

MODULE 1

INTRODUCTION TO DATA STRUCTURES

Q1) Define Data Structure.

A data structure is basically a group of data elements that are put together under one name, and which defines a particular way of storing and organizing data in a computer so that it can be used efficiently.

Data structures are used in almost every program or software system. Some common examples of data structures are arrays, linked lists, queues, stacks, binary trees, and hash tables. Data structures are widely applied in the following areas:

- ❖ Compiler Design
- ❖ Operating System
- ❖ Statistical Analysis Package
- ❖ Numerical Analysis
- ❖ Artificial Intelligence
- ❖ DBMS (Data Base Management Systems)
- ❖ Simulation
- ❖ Graphics

Q2) What are the steps to be followed while selecting a data structure to solve a problem?

When selecting a data structure to solve a problem, the following steps must be performed.

1. Analysis of the problem to determine the basic operations that must be supported. For example, basic operation may include inserting/deleting/searching a data item from the data structure.
2. Quantify the resource constraints for each operation.
3. Select the data structure that best meets these requirements.

Q3) Explain the different types of data structures with an example?

Data structures are generally categorized into two classes: primitive and non-primitive data structures.

Primitive and Non-Primitive Data Structures:

Primitive data structures are the fundamental data types which are supported by a programming language. Some basic data types are integer, real, character, and boolean. The terms ‘data type’, ‘basic data type’, and ‘primitive data type’ are often used interchangeably.

Non-primitive data structures are those data structures which are created using primitive data structures. Examples of such data structures include linked lists, stacks, trees, and graphs.

Non-primitive data structures can further be classified into two categories: linear and non-linear data structures.

Linear and Non-linear Structures:

If the elements of a data structure are stored in a linear or sequential order, then it is a linear data structure. Examples include arrays, linked lists, stacks, and queues. Linear data structures can be represented in memory in two different ways. One way is to have a linear relationship between elements by means of sequential memory locations. The other way is to have a linear relationship between elements by means of links.

However, if the elements of a data structure are not stored in a sequential order, then it is a non-linear data structure. The relationship of adjacency is not maintained between elements of a non-linear data structure. Examples include trees and graphs.

Q4) Discuss the different operations that can be performed on various data structures.

The different operations that can be performed on the various data structures are as follows.

Traversing: It means to access each data item exactly once so that it can be processed. For example, to print the names of all the students in a class.

Searching: It is used to find the location of one or more data items that satisfy the given constraint. Such a data item may or may not be present in the given collection of data items. For example, to find the names of all the students who secured 100 marks in mathematics.

Inserting: It is used to add new data items to the given list of data items. For example, to add the details of a new student who has recently joined the course.

Deleting: It means to remove (delete) a particular data item from the given collection of data items. For example, to delete the name of a student who has left the course.

Sorting: Data items can be arranged in some order like ascending order or descending order depending on the type of application. For example, arranging the names of students in a class in an alphabetical order, or calculating the top three winners by arranging the participants' scores in descending order and then extracting the top three.

Merging: Lists of two sorted data items can be combined to form a single list of sorted data items.

Q5) What is an abstract data type in data structures? What are the advantages of using ADTs?

An abstract data type (ADT) is the way we look at a data structure, focusing on what it does and ignoring how it does its job. For example, stacks and queues are perfect examples of an ADT. We can implement both these ADTs using an array or a linked list.

Data type: Data type of a variable is the set of values that the variable can take. The basic data types used in C are int, char, float, and double.

Abstract: The word ‘abstract’ in the context of data structures means considered apart from the detailed specifications or implementation.

In C, an abstract data type can be a structure considered without regard to its implementation. It can be thought of as a ‘description’ of the data in the structure with a list of operations that can be performed on the data within that structure.

The end-user is not concerned about the details of how the methods carry out their tasks. They are only aware of the methods that are available to them and are only concerned about calling those methods and getting the results. They are not concerned about how they work.

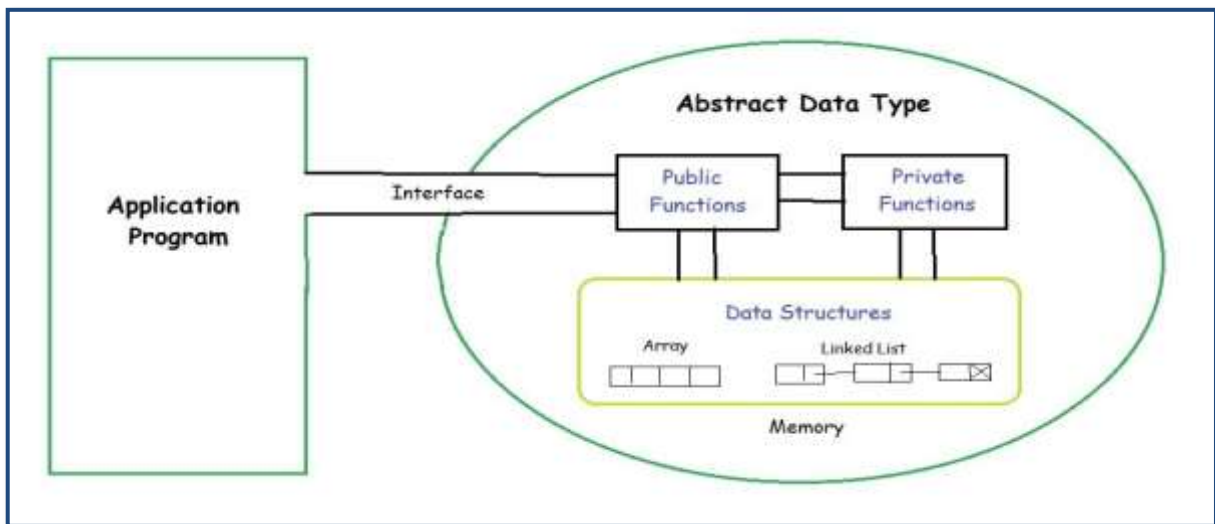
The advantages of using ADTs are:

In the real world, programs evolve as a result of new requirements or constraints, so a modification to a program commonly requires a change in one or more of its data structures. For example, if we want to add a new field to a student’s record to keep track of more information about each student, then it will be better to replace an array with a linked structure to improve the program’s efficiency. In such a scenario, rewriting every procedure that uses the changed structure is not desirable. Therefore, a better alternative is to separate the use of a data structure from the details of its implementation. This is the principle underlying the use of abstract data types.

Q6) Explain the key features of ADT?

Abstract data types (ADTs) are a way of encapsulating data and operations on that data into a single unit. Some of the key features of ADTs include:

- **Abstraction:** The user does not need to know the implementation of the data structure only essentials are provided.
- **Better Conceptualization:** ADT gives us a better conceptualization of the real world.
- **Robust:** The program is robust and has the ability to catch errors.
- **Encapsulation:** ADTs hide the internal details of the data and provide a public interface for users to interact with the data. This allows for easier maintenance and modification of the data structure.
- **Data Abstraction:** ADTs provide a level of abstraction from the implementation details of the data. Users only need to know the operations that can be performed on the data, not how those operations are implemented.
- **Data Structure Independence:** ADTs can be implemented using different data structures, such as arrays or linked lists, without affecting the functionality of the ADT.
- **Information Hiding:** ADTs can protect the integrity of the data by allowing access only to authorized users and operations. This helps prevent errors and misuse of the data.
- **Modularity:** ADTs can be combined with other ADTs to form larger, more complex data structures. This allows for greater flexibility and modularity in programming.



Q7) Write a ADT for Arrays.

structure Array is

objects: A set of pairs where for each value of index there is a value from the set item. Index is a finite ordered set of one or more dimensions, for example, $\{0, \dots, n-1\}$ for one dimension, $\{(0, 0), (0, 1), (0, 2), (1, 0), (1, 1), (1, 2), (2, 0), (2, 1), (2, 2)\}$ for two dimensions, etc.

functions:

for all $A \in \text{Array}$, $i \in \text{index}$, $x \in \text{item}$, j , $\text{size} \in \text{integer}$.

Array Create(j , list) ::= **return** an array of j dimensions where list is a j -tuple whose i^{th} element is the size of the i^{th} dimension. Items are undefined.

Item Retrieve(A , i) ::= **if** ($i \in \text{index}$) **return** the item associated with index value i in array A .
else return error.

Array Store(A, i, x) ::= **if** ($i \in \text{index}$)
return an array that is identical to array A except the new pair $\langle i, x \rangle$ has been inserted.
else return error.

end Array

Q8) What is a sparse matrix? Give the representation of sparse matrix using structures? Give the representation of sparse matrix and its transpose stored as triples?

A sparse matrix is a matrix that has very few non-zero elements spread out thinly. The remaining elements of the matrix are considered to be 0's. Thus, a matrix with few non-zero elements (ie. more number of zeros) is called sparse matrix.

The sparse matrix can be single dimensional or multi-dimensional such as two-dimensional, three-dimensional and so on.

Example 1: Single dimensional matrices

a

10	0	0	0	20	0
0	1	2	3	4	5

 Sparse matrix

b

10	0	30	0	20	40
0	1	2	3	4	5

 Not a Sparse matrix

Example 2: Two-dimensional matrices

A = <table border="1" style="display: inline-table; vertical-align: middle;"><tr><td>-27</td><td>3</td><td>4</td></tr><tr><td>6</td><td>82</td><td>-2</td></tr><tr><td>109</td><td>-64</td><td>11</td></tr><tr><td>12</td><td>8</td><td>9</td></tr><tr><td>48</td><td>27</td><td>47</td></tr></table> Not a Sparse Matrix	-27	3	4	6	82	-2	109	-64	11	12	8	9	48	27	47	B = <table border="1" style="display: inline-table; vertical-align: middle;"><tr><td>15</td><td>0</td><td>0</td><td>22</td><td>0</td><td>-15</td></tr><tr><td>0</td><td>11</td><td>3</td><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>0</td><td>0</td><td>-6</td><td>0</td><td>0</td></tr><tr><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td></tr><tr><td>91</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>0</td><td>28</td><td>0</td><td>0</td><td>0</td></tr></table> Sparse Matrix	15	0	0	22	0	-15	0	11	3	0	0	0	0	0	0	-6	0	0	0	0	0	0	0	0	91	0	0	0	0	0	0	0	28	0	0	0
-27	3	4																																																		
6	82	-2																																																		
109	-64	11																																																		
12	8	9																																																		
48	27	47																																																		
15	0	0	22	0	-15																																															
0	11	3	0	0	0																																															
0	0	0	-6	0	0																																															
0	0	0	0	0	0																																															
91	0	0	0	0	0																																															
0	0	28	0	0	0																																															

Sparse Matrix Representation:

```
#define MAX_TERMS 101
typedef struct
{
    int row, col, value;
} TERM;
TERM a[MAX_TERMS];
```

Representation of Sparse Matrix using triples in a single dimension array:

There are two ways to represent the sparse matrix that are listed as follows -

- Array representation
- Linked list representation

Array representation of the sparse matrix

Representing a sparse matrix by a 2D array leads to the wastage of lots of memory. This is because zeroes in the matrix are of no use, so storing zeroes with non-zero elements is wastage of memory. To avoid such wastage, we can store only non-zero elements. If we store only non-zero elements, it reduces the traversal time and the storage space.

In 2D array representation of sparse matrix, there are three fields used that are named as:

- **Row** - It is the index of a row where a non-zero element is located in the matrix.
- **Column** - It is the index of the column where a non-zero element is located in the matrix.
- **Value** - It is the value of the non-zero element that is located at the index (row, column).

$$A = \begin{bmatrix} 10 & 0 & 0 & 40 \\ 11 & 0 & 22 & 0 \\ 0 & 0 & 0 & 0 \\ 20 & 0 & 0 & -50 \\ 0 & -15 & 0 & 25 \end{bmatrix}$$

Sparse Matrix stored as Triples

	row	col	Value
A[0]	5	4	8
A[1]	0	0	10
A[2]	0	3	40
A[3]	1	0	11
A[4]	1	2	22
A[5]	3	0	20
A[6]	3	3	-50
A[7]	4	1	-15
A[8]	4	3	25

Transpose of Sparse Matrix stored as Triples

	row	col	Value
B[0]	4	5	8
B[1]	0	0	10
B[2]	0	1	11
B[3]	0	3	20
B[4]	1	4	-15
B[5]	2	1	22
B[6]	3	0	40
B[7]	3	3	-50
B[8]	3	4	25

$$B = \begin{bmatrix} 10 & 11 & 0 & 20 & 0 \\ 0 & 0 & 0 & 0 & -15 \\ 0 & 22 & 0 & 0 & 0 \\ 40 & 0 & 0 & -50 & 25 \end{bmatrix}$$

Program 1: Write a program in C to search a given item in a sparse matrix.

```
#define MAX_TERMS 101
#include<stdio.h>
#include<conio.h>
typedef struct
{
    int row; int col; int value;
}TERM;
void main()
{
    TERM a[MAX_TERMS]; int i,key,flag=0;
    clrscr();
    printf("Enter the order of the matrix:");
    scanf("%d%d",&a[0].row,&a[0].col);
    printf("Enter the number of values:");
    scanf("%d",&a[0].value);
    printf("Enter the row number, col number and value:\n");
    for(i=1;i<=a[0].value;i++)
    {
        scanf("%d%d%d",&a[i].row,&a[i].col,&a[i].value);
    }
    printf("The contents of the Sparse Matrix is\n");
    printf("ROW\tCOLUMN\tVALUE\n");
    for(i=0;i<=a[0].value;i++)
    {
        printf("%d\t%d\t%d\n",a[i].row,a[i].col,a[i].value);
    }
    printf("Enter the element to be searched:");
    scanf("%d",&key);
    for(i=1;i<=a[0].value;i++)
    {
        if(key==a[i].value)
        {
            flag=1;
            break;
        }
    }
    if(flag==0)
    {
        printf("Search is unsuccessful\n");
        printf("Key element is not present");
    }
    else
    {
        printf("Search is successful\n");
        printf("Key element is present");
    }
    getch();
}
```

Program 2: Write program in C to transpose a given sparse matrix.

```
#define MAX_TERMS 101
#include<stdio.h>
#include<conio.h>
typedef struct
{
    int row;
    int col;
    int value;
}TERM;
void transpose(TERM[],TERM[]);
void main()
{
    TERM a[MAX_TERMS], b[MAX_TERMS];
    int i,key;
    clrscr();
    printf("Enter the order of the matrix:");
    scanf("%d%d",&a[0].row,&a[0].col);
    printf("Enter the number of values:");
    scanf("%d",&a[0].value);
    printf("Enter the row number, col number and value:\n");
    for(i=1;i<=a[0].value;i++)
    {
        scanf("%d%d%d",&a[i].row,&a[i].col,&a[i].value);
    }
    printf("The contents of the Sparse Matrix is\n");
    printf("ROW\tCOLUMN\tVALUE\n");
    for(i=0;i<=a[0].value;i++)
    {
        printf("%d\t%d\t%d\n",a[i].row,a[i].col,a[i].value);
    }
    transpose(a,b);
    printf("The contents of the Transpose of a Sparse Matrix is\n");
    printf("ROW\tCOLUMN\tVALUE\n");
    for(i=0;i<=b[0].value;i++)
    {
        printf("%d\t%d\t%d\n",b[i].row,b[i].col,b[i].value);
    }
    getch();
}

void transpose(TERM a[MAX_TERMS],TERM b[MAX_TERMS])
{
    int i,j,n,currentb;
    n=a[0].value;
    b[0].row=a[0].col;
    b[0].col=a[0].row;
    b[0].value=n;
```

```
if(n>0)
{
    currentb=1;
    for(i=0;i<a[0].col;i++)
    {
        for(j=1;j<= n;j++)
        {
            if(a[j].col==i)
            {
                b[currentb].row=a[j].col;
                b[currentb].col=a[j].row;
                b[currentb].value=a[j].value;
                currentb++;
            }
        }
    }
}
```

Q9) Write an ADT for Sparse Matrix.

structure Sparse-Matrix is

objects: a set of triples, , where row and column are integers and form a unique combination, and value comes from the set item.

functions:

for all a, be Sparse-Matrix, x e item, i, j, max-col, max-row g index.

Sparse-Matrix Crcate(max-row, max-col) ::= **return** a Sparse-Matrix that can hold up to max—items = max—row X max-col and whose maximum row size is max-row and whose maximum column size is max-col.

Sparse-Matrix Transpose(a) ::= **return** the matrix produced by interchanging the row and column value of every triple.

Sparse-Matrix Add(fl, b) ::= **if** the dimensions of a and b are the same **return** the matrix produced by adding corresponding items, namely those with identical row and column values.

else return error.

Sparse-Matrix Multiply(a, b) ::= **if** number of columns in a equals number of rows in b.

return the matrix d produced by multiplying a by b according to the formula:

$d[i][j] = \sum (a[i][k] \cdot b[k][j])$, where d(i, j) is the (i, j)th element.

else return error.

end Sparse-Matrix.

Q10) Explain the merge sorting technique with an example.

The Merge Sort algorithm is a divide-and-conquer algorithm that sorts an array by first breaking it down into smaller arrays, and then building the array back together the correct way so that it is sorted.

Divide: The algorithm starts with breaking up the array into smaller and smaller pieces until one such sub-array only consists of one element.

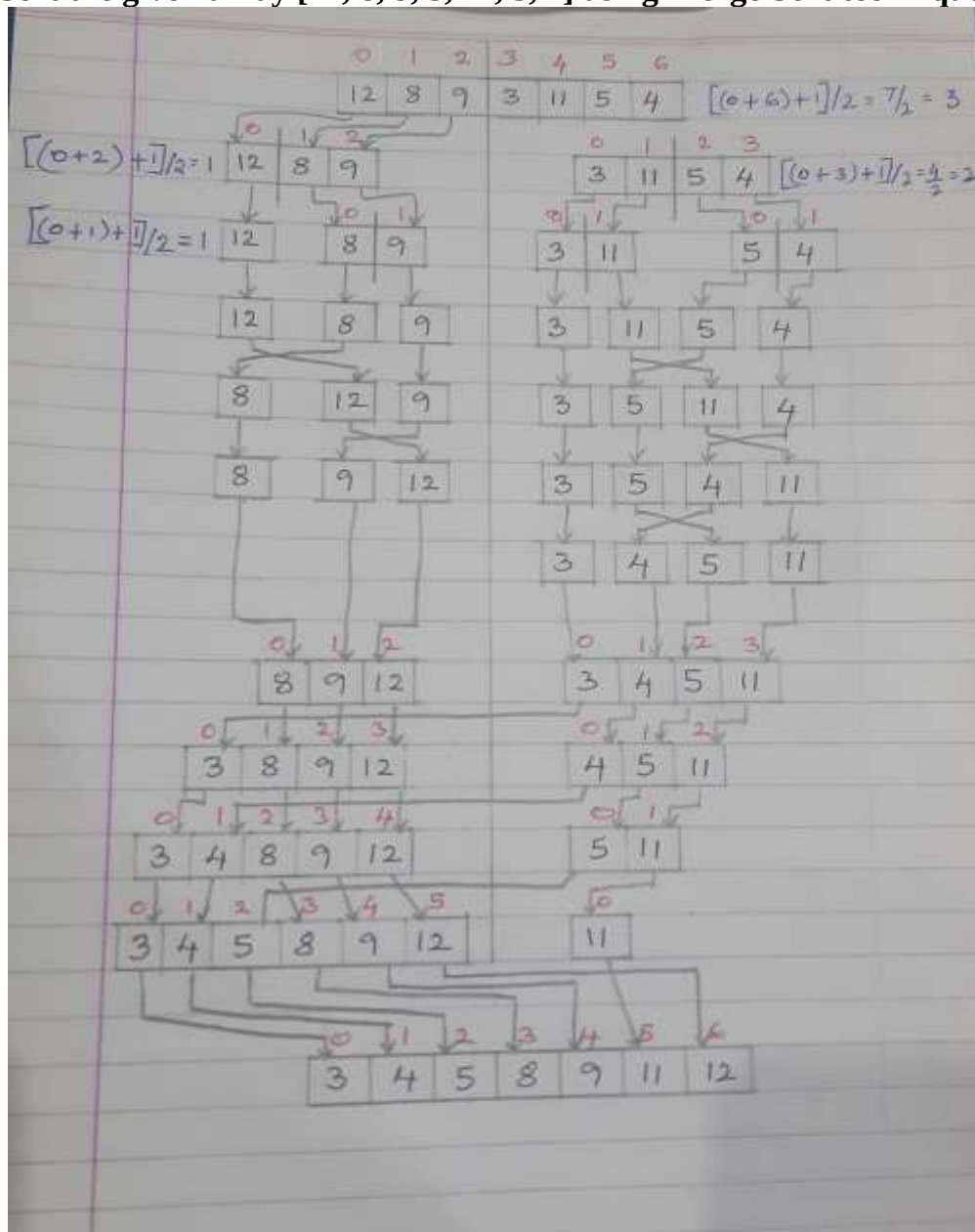
Conquer: The algorithm merges the small pieces of the array back together by putting the lowest values first, resulting in a sorted array.

The breaking down and building up of the array to sort the array is done recursively.

The working of Merge Sort algorithm is described as follows:

1. Divide the unsorted array into two sub-arrays, half the size of the original.
2. Continue to divide the sub-arrays as long as the current piece of the array has more than one element.
3. Merge two sub-arrays together by always putting the lowest value first.
4. Keep merging until there are no sub-arrays left

Example: Sort the given array [12, 8, 9, 3, 11, 5, 4] using Merge Sort technique.



Program 3: Develop a C program to sort the elements of an array using Merge sort technique.

```
#include <stdio.h>
#include <conio.h>
void mergeSort(int arr[], int start, int end);
void merge(int arr[], int start, int mid, int end);
void main()
{
    double unsortedArr[] = {3, 7, 6, -10, 15, 23, 55, -13};
    int size = sizeof(unsortedArr) / sizeof(unsortedArr[0]);
    mergeSort(unsortedArr, 0, size - 1);
    printf("Sorted array: ");
    for (int i = 0; i < size; i++)
    {
        printf("%.1f ", unsortedArr[i]);
    }
    printf("\n");
    getch();
}
void mergeSort(int arr[], int start, int end)
{
    if (start < end)
    {
        int mid = (start + end) / 2;
        mergeSort(arr, start, mid);
        mergeSort(arr, mid + 1, end);
        merge(arr, start, mid, end);
    }
}
void merge(double arr[], int start, int mid, int end)
{
    int i, j, k;
    int n1 = mid - start + 1;
    int n2 = end - mid;
    int left[n1], right[n2];
    for (i = 0; i < n1; i++)
        left[i] = arr[start + i];
    for (j = 0; j < n2; j++)
        right[j] = arr[mid + 1 + j];
    i = 0;
    j = 0;
    k = start;
    while (i < n1 && j < n2)
    {
        if (left[i] <= right[j])
        {
            arr[k] = left[i];
            i++;
        }
        else
```

```
        {
            arr[k] = right[j];
            j++;
        }
        k++;
    }
    while (i < n1)
    {
        arr[k] = left[i];
        i++;
        k++;
    }
    while (j < n2)
    {
        arr[k] = right[j];
        j++;
        k++;
    }
}
```

OUTPUT:

Sorted array: -13, -10, 3, 6, 7, 15, 23, 55

Q11) Explain the Quick Sorting Technique with an example.

Quick sorting technique is one of the fastest sorting algorithms. The Quicksort algorithm takes an array of values as input, chooses one of the values as the '**pivot**' element, and moves the other values so that lower values are on the left of the pivot element, and higher values are on the right of it.

The last element of the array is chosen to be the pivot element, but we could also have chosen the first element of the array, or any element in the array really. Then, the Quicksort algorithm does the same operation recursively on the sub-arrays to the left and right side of the pivot element. This continues until the array is sorted.

After the Quicksort algorithm has put the pivot element in between a sub-array with lower values on the left side, and a sub-array with higher values on the right side, the algorithm calls itself twice, so that Quicksort runs again for the sub-array on the left side, and for the sub-array on the right side. The Quicksort algorithm continues to call itself until the sub-arrays are too small to be sorted.

Working of Quick Sort Technique:

1. Choose a value in the array to be the pivot element.
2. Order the rest of the array so that lower values than the pivot element are on the left, and higher values are on the right.
3. Swap the pivot element with the first element of the higher values so that the pivot element lands in between the lower and higher values.
4. Do the same operations (recursively) for the sub-arrays on the left and right side of the pivot element.

Example: Sort the given array [11, 9, 12, 7, 3] using Quick Sort technique.

Step 1: Start with an unsorted array.

[11, 9, 12, 7, 3]

Step 2: We choose the last value 3 as the pivot element.

[11, 9, 12, 7, 3]

Step 3: The rest of the values in the array are all greater than 3, and must be on the right side of 3. Swap 3 with 11.

[3, 9, 12, 7, 11]

Step 4: Value 3 is now in the correct position. We need to sort the values to the right of 3. Choose the last value 11 as the new pivot element.

[3, 9, 12, 7, 11]

Step 5: The value 7 must be to the left of pivot value 11, and 12 must be to the right of it. Move 7 and 12.

[3, 9, 7, 12, 11]

Step 6: Swap 11 with 12 so that lower values 9 and 7 are on the left side of 11, and 12 is on the right side.

[3, 9, 7, 11, 12]

Step 7: 11 and 12 are in the correct positions. Choose 7 as the pivot element in sub-array [9, 7], to the left of 11.

[3, 9, 7, 11, 12]

Step 8: Swap 9 with 7.

[3, 7, 9, 11, 12]

Q12) Explain the Radix Sort technique with an example.

The radix (or base) is the number of unique digits in a number system. In the decimal system we normally use, there are 10 different digits from 0 till 9. Radix Sort uses the radix so that decimal values are put into 10 different buckets (or containers) corresponding to the digit that is in focus, then put back into the array before moving on to the next digit.

Note: Radix Sort is a non-comparative algorithm that only works with non-negative integers.

The Radix Sort algorithm can be described like this:

Radix Sort Implementation:

The implementation of Radix Sort algorithm is as follows:

1. An array with non-negative integers that needs to be sorted.
2. A two dimensional array with index 0 to 9 to hold values with the current radix in focus.
3. A loop that takes values from the unsorted array and places them in the correct position in the two dimensional radix array.
4. A loop that puts values back into the initial array from the radix array.
5. An outer loop that runs as many times as there are digits in the highest value.

Example: Sort the given array [33, 45, 40, 25, 17, 24] using Radix Sort technique.

Step 1: We start with an unsorted array, and an empty array to fit values with corresponding radices 0 till 9.

myArray = [33, 45, 40, 25, 17, 24]

radixArray = [[], [], [], [], [], [], [], [], [], []]

Step 2: We start sorting by focusing on the least significant digit.

myArray = [33, 45, 40, 25, 17, 24]

radixArray = [[], [], [], [], [], [], [], [], [], []]

Step 3: Now we move the elements into the correct positions in the radix array according to the digit in focus. Elements are taken from the start of myArray and pushed into the correct position in the radixArray.

myArray = []

radixArray = [[40], [], [], [33], [24], [45, 25], [], [17], [], []]

Step 4: We move the elements back into the initial array, and the sorting is now done for the least significant digit. Elements are taken from the end radixArray, and put into the start of myArray.

myArray = [40, 33, 24, 45, 25, 17]

radixArray = [[], [], [], [], [], [], [], [], [], []]

Step 5: We move focus to the next digit. Notice that values 45 and 25 are still in the same order relative to each other as they were to start with, because we sort in a stable way.

myArray = [40, 33, 24, 45, 25, 17]

radixArray = [[], [], [], [], [], [], [], [], [], []]

Step 6: We move elements into the radix array according to the focused digit.

myArray = []

radixArray = [[], [17], [24, 25], [33], [40, 45], [], [], [], [], []]

Step 7: We move elements back into the start of myArray, from the back of radixArray.

myArray = [17, 24, 25, 33, 40, 45]

radixArray = [[], [], [], [], [], [], [], [], [], []]

The sorting is finished.

Program 4: Develop a C program to sort the elements of an array using Radix sort technique.

```
#include<stdio.h>
#include<conio.h>
int get_max (int a[], int n)
{
    int max = a[0];
    for (i = 1; i < n; i++)
        if (a[i] > max)
            max = a[i];
    return max;
}
void radix_sort (int a[], int n)
{
    int bucket[10][10], bucket_cnt[10];
    int i, j, k, r, NOP = 0, divisor = 1, lar, pass;
    lar = get_max (a, n);
    while (lar > 0)
    {
        NOP++;
        lar /= 10;
    }
    for (pass = 0; pass < NOP; pass++)
    {
        for (i = 0; i < 10; i++)
        {
            bucket_cnt[i] = 0;
        }
        for (i = 0; i < n; i++)
        {
            r = (a[i] / divisor) % 10;
            bucket[r][bucket_cnt[r]] = a[i];
            bucket_cnt[r] += 1;
        }
        i = 0;
        for (k = 0; k < 10; k++)
        {
            for (j = 0; j < bucket_cnt[k]; j++)
            {
                a[i] = bucket[k][j];
                i++;
            }
        }
        divisor *= 10;
        printf ("\n After pass %d : ", pass + 1);
        for (i = 0; i < n; i++)
            printf ("\n %d", a[i]);
    }
}
```

```
void main ()
{
    int i, n, a[10];
    clrscr();
    printf ("\n Enter the number of elements to be sorted : ");
    scanf ("%d", &n);
    printf ("\n Enter the elements : ");
    for (i = 0; i < n; i++)
    {
        scanf ("%d",&a[i]);
    }
    radix_sort (a, n);
    printf ("\n Sorted items : ");
    for (i = 0; i < n; i++)
        printf ("%d\t",a[i]);
    getch();
}
```

Q13) Explain Insertion sort technique with an example.

Insertion sort is a simple sorting algorithm that builds a sorted array one element at a time. It works by iteratively taking elements from the unsorted part of the array and inserting them into their correct position within the already sorted portion.

The insertion sort algorithm works as follows:

- To sort an array of size N in ascending order, iterate over the array.
- For each element (referred to as the "key"), compare it to its preceding element.
- If the key element is smaller than its predecessor, continue comparing it to the preceding elements.
- Shift the larger elements one position forward to make room for the key element.
- Repeat the process until the array is fully sorted.

Example: Sort the given array [4, 3, 2, 10, 12, 1, 5, 6] using insertion sort technique.



Q14) What is hashing? What are different types of hashing techniques? Construct the hash table for storing C built-in functions.

acos, define, float, exp, char, atan, ceil, floor

Note: Use hash table with 26 buckets and two slots per bucket.

The process of mapping large amounts of data into a smaller table using hash function, hash value, and hash table is called hashing.

The two types of hashing techniques are:

- i) Static hashing
- ii) Dynamic hashing

The process of mapping large amounts of data into a table whose size is fixed during compilation time is called static hashing.

The process of mapping large amounts of data into a table whose size is fixed during run time is called static hashing.

	Slot 0	Slot 1
0	acos	Atan
1		
2		
3	char	Ceil
4	define	
5	Exp	
6	float	floor
...		
25		

Figure: Hash table with 26 buckets and two slots per bucket

Q15) Define hash function? What are the properties of a good hash function? Explain the different hash functions with an example.

A hash function is a mathematical formula which, when applied to a key, produces an integer which can be used as an index for the key in the hash table. The main aim of a hash function is that elements should be relatively, randomly, and uniformly distributed. It produces a unique set of integers within some suitable range in order to reduce the number of collisions. A good hash function can only minimize the number of collisions by spreading the elements uniformly throughout the array.

The properties of a good hash function are as follows.

- 1) Low Cost:** The cost of executing a hash function must be small, so that using the hashing technique becomes preferable over other approaches.
- 2) Determinism:** A hash procedure must be deterministic. This means that the same hash value must be generated for a given input value.
- 3) Uniformity:** A good hash function must map the keys as evenly as possible over its output range. This means that the probability of generating every hash value in the output range should roughly be the same. The property of uniformity also minimizes the number of collisions.

The different hash functions are as follows.

- i) Division Method.
- ii) Multiplication Method.
- iii) Mid-square Method.
- iv) Folding Method.

i) Division Method:

It is the most simple method of hashing an integer x . This method divides x by M and then uses the remainder obtained.

In division method, the hash function can be given as

$$h(x) = x \bmod M$$

where x is the key and M is the hash table size.

Note: If M is an even number then $h(x)$ is even if x is even and $h(x)$ is odd if x is odd.

Example 1: Calculate the hash value of key $x = 1234$ and $M = 10$

(If M is an even number then $h(x)$ is even if x is even)

$$h(x) = x \bmod M$$

$$h(1234) = 1234 \bmod 10 = 4 \quad (4 \text{ is even})$$

Example 2: Calculate the hash value of key $x = 5463$ and $M = 10$

(If M is an even number then $h(x)$ is odd if x is odd)

$$h(x) = x \bmod M$$

$$h(5463) = 5463 \bmod 10 = 3 \quad (3 \text{ is odd})$$

ii) Multiplication Method:

The steps involved in the multiplication method are as follows:

Step 1: Choose a constant A such that $0 < A < 1$

Step 2: Multiply the key k by A .

Step 3: Extract the fractional part of kA .

Step 4: Multiply the result of Step 3 by the size of hash table (m).

Hence, the hash function can be given as:

$$h(k) = \lfloor m (kA \bmod 1) \rfloor$$

Example: Given the hash table of size 1000, map the key 12345 to an appropriate location in the hash table.

Choose $A = 0.3456789$, $m = 1000$ and $k = 12345$

$$\begin{aligned}h(12345) &= \lfloor 1000 (12345 * 0.3456789 \bmod 1) \rfloor \\&= \lfloor 1000 (4267.4060205 \bmod 1) \rfloor \\&= \lfloor 1000 (0.4060205) \rfloor \\&= \lfloor 406.0205 \rfloor \\&= 406\end{aligned}$$

iii) Mid-Square Method:

The mid-square method is a good hash function which works in two steps:

Step 1: Square the value of the key. That is, find k^2 .

Step 2: Extract the middle r digits of the result obtained in Step 1.

In the mid-square method, the same r digits must be chosen from all the keys. Therefore, the hash function can be given as:

$$h(k) = s$$

where s is obtained by selecting r digits from k^2 .

Example: Calculate the hash value for keys 1234 and 5642 using the mid-square method. The hash table has 100 memory locations.

Note: The hash table has 100 memory locations whose indices vary from 0 to 99. This means that only two digits are needed to map the key to a location in the hash table, so $r = 2$.

When $k = 1234$, $k^2 = 1522756$, $h(1234) = 27$

1	5	2	2	7	5	6
---	---	---	---	---	---	---

Remove 1 5 and 5 6

2	2	7
---	---	---

(In 227, 7 is in unit position, 2 is on tens position and 2 is in hundredth position. Memory locations, whose indices are vary from 0 to 99. This means that only two digits are needed to map the key to a location in the hash table, so $r = 2$. In the result we have to take value from unit position and tenth position and the answer is 27)

When $k = 5642$, $k^2 = 31832164$, $h(5642) = 32$

iv) Folding Method:

The folding method works in the following two steps:

Step 1: Divide the key value into a number of parts. That is, divide k into parts $k_1, k_2, \dots, k_n$, where each part has the same number of digits except the last part which may have lesser digits than the other parts.

Step 2: Add the individual parts. That is, obtain the sum of $k_1 + k_2 + \dots + k_n$. The hash value is produced by ignoring the last carry, if any.

Example: Given a hash table of 100 locations, calculate the hash value using folding method for keys 5678, 321, and 34567.

Since there are 100 memory locations to address, we will break the key into parts where each part (except the last) will contain two digits. The hash values can be obtained as shown below:

key	5678	321	34567
Parts	56 and 78	32 and 1	34, 56 and 7
Sum	134	33	97
Hash value	34 (ignore the last carry)	33	97

Q16) Define and explain Hash collision with an example?

A hash collision is when two different inputs produce the same output hash value from a hash function, leading to a conflict in data structures like hash tables where multiple inputs map to the same index or location.

Example 1 of a Simple Hash Collision:

Consider a very simple hash function that calculates the sum of the ASCII values of the characters in a string, and then takes the result modulo a small number (e.g., 10) to determine an index in a hash table.

hash function: $\text{hash}(\text{string}) = (\text{sum of ASCII values of characters}) \% 10$

- **Input 1: "cat"**
 ASCII values: 'c' = 99, 'a' = 97, 't' = 116
 Sum: $99 + 97 + 116 = 312$
 Hash value: $312 \% 10 = 2$
- **Input 2: "act"**
 ASCII values: 'a' = 97, 'c' = 99, 't' = 116
 Sum: $97 + 99 + 116 = 312$
 Hash value: $312 \% 10 = 2$

In this example, both "cat" and "act" produce the same hash value of 2, even though they are distinct strings. This demonstrates a hash collision.

Example 2 of a Simple Hash Collision:

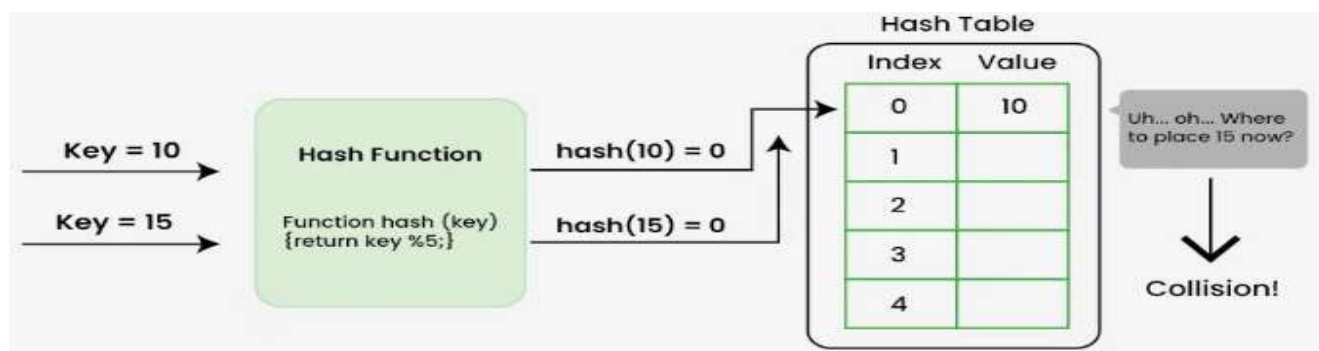


Figure: Collision in Hashing

Q17) List and explain different collision resolution techniques.

Collisions occur when the hash function maps two different keys to the same location. Two records cannot be stored in the same location. Therefore, a method used to solve the problem of collision, also called collision resolution technique, is applied. The two most popular methods of resolving collisions are:

1. Open addressing (or Closed Hashing)
2. Chaining

Open Addressing:

Open addressing or closed hashing computes new positions using a probe sequence and the next record is stored in that position. In this technique, all the values are stored in the hash table. The hash table contains two types of values: sentinel values (e.g., -1) and data values. The presence of a sentinel value indicates that the location contains no data value at present but can be used to hold a value.

When a key is mapped to a particular memory location, then the value it holds is checked. If it contains a sentinel value, then the location is free and the data value can be stored in it. However, if the location already has some data value stored in it, then other slots are examined systematically in the forward direction to find a free slot. If even a single free location is not found, then we have an OVERFLOW condition.

The process of examining memory locations in the hash table is called probing. Open addressing technique can be implemented using following techniques.

- 1) Linear Probing
- 2) Quadratic Probing
- 3) Double Hashing.

Linear Probing:

The simplest approach to resolve a collision is linear probing. In this technique, if a value is already stored at a location generated by $h(k)$, then the following hash function is used to resolve the collision:

$$h(k, i) = [h(k) + i] \bmod m$$

Where m is the size of the hash table, $h(k) = (k \bmod m)$, and i is the probe number that varies from 0 to $m-1$.

Note: Linear probing is known for its simplicity. When we want to store a value, we have to try the slots:

$[h(k)] \bmod m$, $[h'(k) + 1] \bmod m$, $[h'(k) + 2] \bmod m$, $[h'(k) + 3] \bmod m$, $[h'(k) + 4] \bmod m$, $[h'(k) + 5] \bmod m$, and so on, until a vacant location is found.

Quadratic Probing:

In this technique, if a value is already stored at a location generated by $h(k)$, then the following hash function is used to resolve the collision:

$$h(k, i) = [h'(k) + c_1 i + c_2 i^2] \bmod m$$

where m is the size of the hash table, $h'(k) = (k \bmod m)$, i is the probe number that varies from 0 to $m-1$, and c_1 and c_2 are constants such that c_1 and $c_2 \neq 0$.

Double Hashing:

Double hashing uses one hash value and then repeatedly steps forward an interval until an empty location is reached. The interval is decided using a second, independent hash function, hence the name double hashing. In double hashing, we use two hash functions rather than a single function. The hash function in the case of double hashing can be given as:

$$h(k, i) = [h_1(k) + ih_2(k)] \bmod m$$

where m is the size of the hash table, $h_1(k)$ and $h_2(k)$ are two hash functions given as $h_1(k) = k \bmod m$, $h_2(k) = k \bmod m'$, i is the probe number that varies from 0 to $m-1$, and m' is chosen to be less than m . We can choose $m' = m-1$ or $m-2$.

Collision Resolution by Chaining:

In chaining, each location in a hash table stores a pointer to a linked list that contains all the key values that were hashed to that location. That is, location l in the hash table points to the head of the linked list of all the key values that hashed to l . However, if no key value hashes to l , then location l in the hash table contains NULL. Figure below shows how the key values are mapped to a location in the hash table and stored in a linked list that corresponds to that location

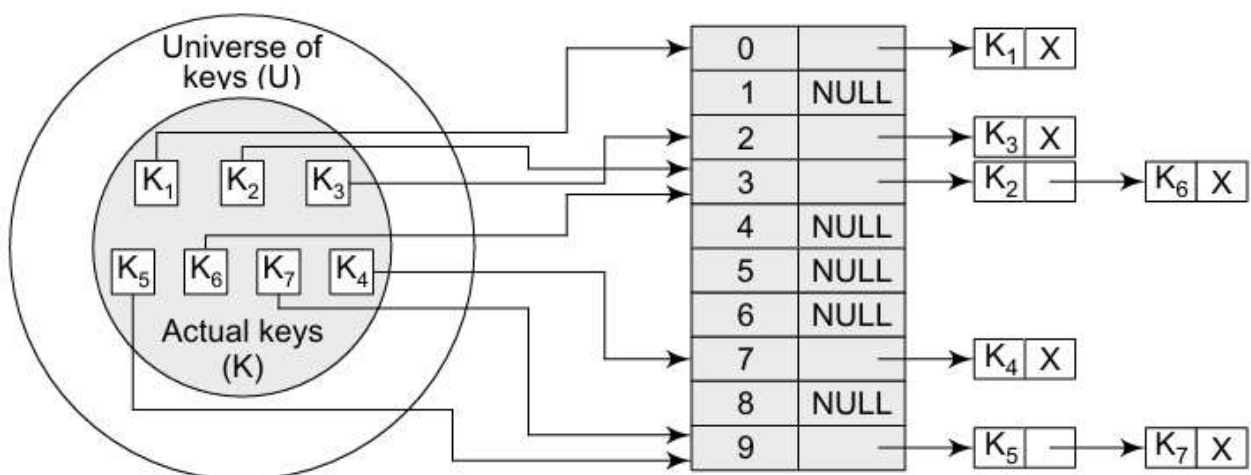


Figure Keys being hashed to a chained hash table

Q18) Consider a hash table of size 10. Using Linear Probing, insert the keys 72, 27, 36, 24, 63, 81, 92, and 101 into the table.

Let $h'(k) = k \bmod m, m = 10$

Initially, the hash table can be given as:

0	1	2	3	4	5	6	7	8	9
-1	-1	-1	-1	-1	-1	-1	-1	-1	-1

Step 1 Key = 72

$$\begin{aligned}h(72, 0) &= (72 \bmod 10 + 0) \bmod 10 \\&= (2) \bmod 10 \\&= 2\end{aligned}$$

Since $\tau[2]$ is vacant, insert key 72 at this location.

0	1	2	3	4	5	6	7	8	9
-1	-1	72	-1	-1	-1	-1	-1	-1	-1

Step 2 Key = 27

$$\begin{aligned}h(27, 0) &= (27 \bmod 10 + 0) \bmod 10 \\&= (7) \bmod 10 \\&= 7\end{aligned}$$

Since $\tau[7]$ is vacant, insert key 27 at this location.

0	1	2	3	4	5	6	7	8	9
-1	-1	72	-1	-1	-1	-1	27	-1	-1

0	1	2	3	4	5	6	7	8	9
-1	-1	72	-1	-1	-1	-1	27	-1	-1

Step 3 Key = 36

$$\begin{aligned}h(36, 0) &= (36 \bmod 10 + 0) \bmod 10 \\&= (6) \bmod 10 \\&= 6\end{aligned}$$

Since $\tau[6]$ is vacant, insert key 36 at this location.

0	1	2	3	4	5	6	7	8	9
-1	-1	72	-1	-1	-1	36	27	-1	-1

Step 4 Key = 24

$$h(24, 0) = (24 \bmod 10 + 0) \bmod 10$$

$$= (4) \bmod 10$$

$$= 4$$

Since $\tau[4]$ is vacant, insert key 24 at this location.

0	1	2	3	4	5	6	7	8	9
-1	-1	72	-1	24	-1	36	27	-1	-1

Step 5

Key = 63

$$h(63, 0) = (63 \bmod 10 + 0) \bmod 10$$

$$= (3) \bmod 10$$

$$= 3$$

Since $\tau[3]$ is vacant, insert key 63 at this location.

0	1	2	3	4	5	6	7	8	9
-1	-1	72	63	24	-1	36	27	-1	-1

Step 6

Key = 81

$$h(81, 0) = (81 \bmod 10 + 0) \bmod 10$$

$$= (1) \bmod 10$$

$$= 1$$

Since $\tau[1]$ is vacant, insert key 81 at this location.

0	1	2	3	4	5	6	7	8	9
0	81	72	63	24	-1	36	27	-1	-1

Step 7

Key = 92

$$h(92, 0) = (92 \bmod 10 + 0) \bmod 10$$

$$= (2) \bmod 10$$

$$= 2$$

Now $\tau[2]$ is occupied, so we cannot store the key 92 in $\tau[2]$. Therefore, try again for the next location. Thus probe, $i = 1$, this time.

Key = 92

$$h(92, 1) = (92 \bmod 10 + 1) \bmod 10$$

$$= (2 + 1) \bmod 10$$

$$= 3$$

Now $\tau[3]$ is occupied, so we cannot store the key 92 in $\tau[3]$. Therefore, try again for the next location. Thus probe, $i = 2$, this time.

Key = 92

$$h(92, 2) = (92 \bmod 10 + 2) \bmod 10$$

$$= (2 + 2) \bmod 10$$

$$= 4$$

Now $\tau[4]$ is occupied, so we cannot store the key 92 in $\tau[4]$. Therefore, try again for the next location. Thus probe, $i = 3$, this time.

Key = 92

$$h(92, 3) = (92 \bmod 10 + 3) \bmod 10$$

$$= (2 + 3) \bmod 10$$

$$= 5$$

Since $\tau[5]$ is vacant, insert key 92 at this location.

0	1	2	3	4	5	6	7	8	9
-1	81	72	63	24	92	36	27	-1	-1

Step 8

Key = 101

$$\begin{aligned}h(101, 0) &= (101 \bmod 10 + 0) \bmod 10 \\&= (1) \bmod 10 \\&= 1\end{aligned}$$

Now $\tau[1]$ is occupied, so we cannot store the key 101 in $\tau[1]$. Therefore, try again for the next location. Thus probe, $i = 1$, this time.

Key = 101

$$\begin{aligned}h(101, 1) &= (101 \bmod 10 + 1) \bmod 10 \\&= (1 + 1) \bmod 10 \\&= 2\end{aligned}$$

$\tau[2]$ is also occupied, so we cannot store the key in this location. The procedure will be repeated until the hash function generates the address of location 8 which is vacant and can be used to store the value in it.

Q19) Consider a hash table of size 10. Using Quadratic Probing, insert the keys 72, 27, 36, 24, 63, 81, and 101 into the table. Take $c_1 = 1$ and $c_2 = 3$.

Let $h'(k) = k \bmod m$, $m = 10$

Initially, the hash table can be given as:

0	1	2	3	4	5	6	7	8	9
-1	-1	-1	-1	-1	-1	-1	-1	-1	-1

We have,

$$h(k, i) = [h'(k) + c_1 i + c_2 i^2] \bmod m$$

Step 1

Key = 72

$$\begin{aligned}h(72, 0) &= [72 \bmod 10 + 1 \times 0 + 3 \times 0] \bmod 10 \\&= [72 \bmod 10] \bmod 10 \\&= 2 \bmod 10 \\&= 2\end{aligned}$$

Since $\tau[2]$ is vacant, insert the key 72 in $\tau[2]$. The hash table now becomes:

0	1	2	3	4	5	6	7	8	9
-1	-1	72	-1	-1	-1	-1	-1	-1	-1

Step 2

Key = 27

$$\begin{aligned}h(27, 0) &= [27 \bmod 10 + 1 \times 0 + 3 \times 0] \bmod 10 \\&= [27 \bmod 10] \bmod 10 \\&= 7 \bmod 10 \\&= 7\end{aligned}$$

Since $\tau[7]$ is vacant, insert the key 27 in $\tau[7]$. The hash table now becomes:

0	1	2	3	4	5	6	7	8	9
-1	-1	72	-1	-1	-1	-1	27	-1	-1

Step 3 Key = 36

$$\begin{aligned}h(36, 0) &= [36 \bmod 10 + 1 \times 0 + 3 \times 0] \bmod 10 \\&= [36 \bmod 10] \bmod 10 \\&= 6 \bmod 10 \\&= 6\end{aligned}$$

Since $\tau[6]$ is vacant, insert the key 36 in $\tau[6]$. The hash table now becomes:

0	1	2	3	4	5	6	7	8	9
-1	-1	72	-1	-1	-1	36	27	-1	-1

Step 4 Key = 24

$$\begin{aligned}h(24, 0) &= [24 \bmod 10 + 1 \times 0 + 3 \times 0] \bmod 10 \\&= [24 \bmod 10] \bmod 10 \\&= 4 \bmod 10 \\&= 4\end{aligned}$$

Since $\tau[4]$ is vacant, insert the key 24 in $\tau[4]$. The hash table now becomes:

0	1	2	3	4	5	6	7	8	9
-1	-1	72	-1	24	-1	36	27	-1	-1

Step 5 Key = 63

$$\begin{aligned}h(63, 0) &= [63 \bmod 10 + 1 \times 0 + 3 \times 0] \bmod 10 \\&= [63 \bmod 10] \bmod 10 \\&= 3 \bmod 10 \\&= 3\end{aligned}$$

Since $\tau[3]$ is vacant, insert the key 63 in $\tau[3]$. The hash table now becomes:

0	1	2	3	4	5	6	7	8	9
-1	-1	72	63	24	-1	36	27	-1	-1

Step 6 Key = 81

$$\begin{aligned}h(81, 0) &= [81 \bmod 10 + 1 \times 0 + 3 \times 0] \bmod 10 \\&= [81 \bmod 10] \bmod 10 \\&= 1 \bmod 10 \\&= 1\end{aligned}$$

Since $\tau[1]$ is vacant, insert the key 81 in $\tau[1]$. The hash table now becomes:

0	1	2	3	4	5	6	7	8	9
-1	81	72	63	24	-1	36	27	-1	-1

Step 7 Key = 101

$$\begin{aligned}h(101, 0) &= [101 \bmod 10 + 1 \times 0 + 3 \times 0] \bmod 10 \\&= [101 \bmod 10 + 0] \bmod 10 \\&= 1 \bmod 10 \\&= 1\end{aligned}$$

Since $\tau[1]$ is already occupied, the key 101 cannot be stored in $\tau[1]$. Therefore, try again for next location. Thus probe, $i = 1$, this time.

Key = 101

$$\begin{aligned}h(101, 0) &= [101 \bmod 10 + 1 \times 1 + 3 \times 1] \bmod 10 \\&= [101 \bmod 10 + 1 + 3] \bmod 10 \\&= [101 \bmod 10 + 4] \bmod 10 \\&= [1 + 4] \bmod 10 \\&= 5 \bmod 10 \\&= 5\end{aligned}$$

Since $\tau[5]$ is vacant, insert the key 101 in $\tau[5]$. The hash table now becomes:

0	1	2	3	4	5	6	7	8	9
-1	81	72	63	24	101	36	27	-1	-1

Q20) Consider a hash table of size = 10. Using Double Hashing, insert the keys 72, 27, 36, 24, 63, 81, 92, and 101 into the table. Take $h_1 = (k \bmod 10)$ and $h_2 = (k \bmod 8)$.

Let $m = 10$

Initially, the hash table can be given as:

0	1	2	3	4	5	6	7	8	9
-1	-1	-1	-1	-1	-1	-1	-1	-1	-1

We have,

$$h(k, i) = [h_1(k) + ih_2(k)] \bmod m$$

Step 1 Key = 72

$$\begin{aligned}h(72, 0) &= [72 \bmod 10 + (0 \times 72 \bmod 8)] \bmod 10 \\&= [2 + (0 \times 0)] \bmod 10 \\&= 2 \bmod 10 \\&= 2\end{aligned}$$

Since $\tau[2]$ is vacant, insert the key 72 in $\tau[2]$. The hash table now becomes:

0	1	2	3	4	5	6	7	8	9
-1	-1	72	-1	-1	-1	-1	-1	-1	-1

Step 2 Key = 27

$$\begin{aligned}h(27, 0) &= [27 \bmod 10 + (0 \times 27 \bmod 8)] \bmod 10 \\&= [7 + (0 \times 3)] \bmod 10 \\&= 7 \bmod 10 \\&= 7\end{aligned}$$

Since $\tau[7]$ is vacant, insert the key 27 in $\tau[7]$. The hash table now becomes:

0	1	2	3	4	5	6	7	8	9
-1	-1	72	-1	-1	-1	-1	27	-1	-1

Step 3 Key = 36

$$\begin{aligned}h(36, 0) &= [36 \bmod 10 + (0 \times 36 \bmod 8)] \bmod 10 \\&= [6 + (0 \times 4)] \bmod 10 \\&= 6 \bmod 10 \\&= 6\end{aligned}$$

Since $\tau[6]$ is vacant, insert the key 36 in $\tau[6]$. The hash table now becomes:

0	1	2	3	4	5	6	7	8	9
-1	-1	72	-1	-1	-1	36	27	-1	-1

Step 4 Key = 24

$$\begin{aligned}h(24, 0) &= [24 \bmod 10 + (0 \times 24 \bmod 8)] \bmod 10 \\&= [4 + (0 \times 0)] \bmod 10 \\&= 4 \bmod 10 \\&= 4\end{aligned}$$

Since $\tau[4]$ is vacant, insert the key 24 in $\tau[4]$. The hash table now becomes:

0	1	2	3	4	5	6	7	8	9
-1	-1	72	-1	24	-1	36	27	-1	-1

Step 5 Key = 63

$$\begin{aligned}h(63, 0) &= [63 \bmod 10 + (0 \times 63 \bmod 8)] \bmod 10 \\&= [3 + (0 \times 7)] \bmod 10 \\&= 3 \bmod 10 \\&= 3\end{aligned}$$

Since $\tau[3]$ is vacant, insert the key 63 in $\tau[3]$. The hash table now becomes:

0	1	2	3	4	5	6	7	8	9
-1	-1	72	63	24	-1	36	27	-1	-1

Step 6 Key = 81

$$\begin{aligned}h(81, 0) &= [81 \bmod 10 + (0 \times 81 \bmod 8)] \bmod 10 \\&= [1 + (0 \times 1)] \bmod 10 \\&= 1 \bmod 10 \\&= 1\end{aligned}$$

Since $\tau[1]$ is vacant, insert the key 81 in $\tau[1]$. The hash table now becomes:

0	1	2	3	4	5	6	7	8	9
-1	81	72	63	24	-1	36	27	-1	-1

Step 7 Key = 92

$$\begin{aligned}h(92, 0) &= [92 \bmod 10 + (0 \times 92 \bmod 8)] \bmod 10 \\&= [2 + (0 \times 4)] \bmod 10 \\&= 2 \bmod 10 \\&= 2\end{aligned}$$

Now $\tau[2]$ is occupied, so we cannot store the key 92 in $\tau[2]$. Therefore, try again for the next location. Thus probe, $i = 1$, this time.

Key = 92

$$\begin{aligned}h(92, 1) &= [92 \bmod 10 + (1 \times 92 \bmod 8)] \bmod 10 \\&= [2 + (1 \times 4)] \bmod 10 \\&= (2 + 4) \bmod 10 \\&= 6 \bmod 10 \\&= 6\end{aligned}$$

Now $\tau[6]$ is occupied, so we cannot store the key 92 in $\tau[6]$. Therefore, try again for the next location. Thus probe, $i = 2$, this time.

Key = 92

$$\begin{aligned}h(92, 2) &= [92 \bmod 10 + (2 \times 92 \bmod 8)] \bmod 10 \\&= [2 + (2 \times 4)] \bmod 10 \\&= [2 + 8] \bmod 10 \\&= 10 \bmod 10 \\&= 0\end{aligned}$$

Since $\tau[0]$ is vacant, insert the key 92 in $\tau[0]$. The hash table now becomes:

0	1	2	3	4	5	6	7	8	9
92	81	72	63	24	-1	36	27	-1	-1

Step 8 Key = 101

$$\begin{aligned}h(101, 0) &= [101 \bmod 10 + (0 \times 101 \bmod 8)] \bmod 10 \\&= [1 + (0 \times 5)] \bmod 10 \\&= 1 \bmod 10 \\&= 1\end{aligned}$$

Now $\tau[1]$ is occupied, so we cannot store the key 101 in $\tau[1]$. Therefore, try again for the next location. Thus probe, $i = 1$, this time.

Key = 101

$$\begin{aligned}h(101, 1) &= [101 \bmod 10 + (1 \times 101 \bmod 8)] \bmod 10 \\&= [1 + (1 \times 5)] \bmod 10 \\&= [1 + 5] \bmod 10 \\&= 6\end{aligned}$$

Q21) Insert the keys 7, 24, 18, 52, 36, 54, 11, and 23 in a chained hash table of 9 memory locations. Use $h(k) = k \bmod m$.

In this case, $m=9$. Initially, the hash table can be given as:

0	NULL
1	NULL
2	NULL
3	NULL
4	NULL
5	NULL
6	NULL
7	NULL
8	NULL

Step 1 Key = 7

$$\begin{aligned}h(k) &= 7 \bmod 9 \\&= 7\end{aligned}$$

Create a linked list for location 7 and store the key value 7 in it as its only node.

0	NULL
1	NULL
2	NULL
3	NULL
4	NULL
5	NULL
6	NULL
7	→ [7 X]
8	NULL

Step 2 Key = 24

$$\begin{aligned}h(k) &= 24 \bmod 9 \\&= 6\end{aligned}$$

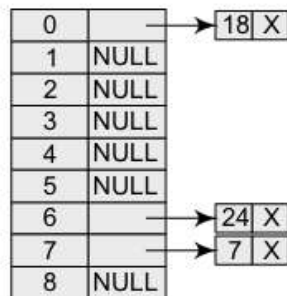
Create a linked list for location 6 and store the key value 24 in it as its only node.

0	NULL
1	NULL
2	NULL
3	NULL
4	NULL
5	NULL
6	→ [24 X]
7	→ [7 X]
8	NULL

Step 3 Key = 18

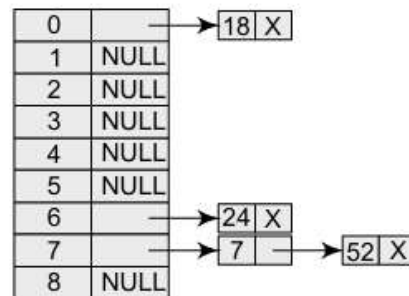
$$h(k) = 18 \bmod 9 = 0$$

Create a linked list for location 0 and store the key value 18 in it as its only node.

**Step 4** Key = 52

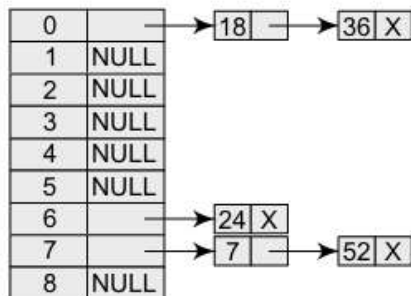
$$h(k) = 52 \bmod 9 = 7$$

Insert 52 at the end of the linked list of location 7.

**Step 5:** Key = 36

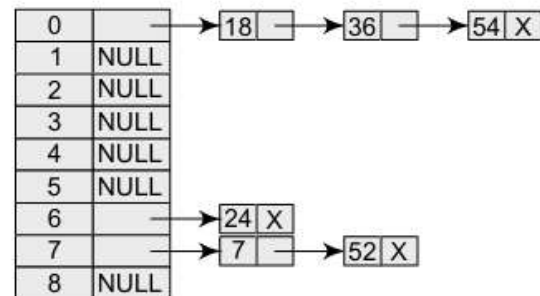
$$h(k) = 36 \bmod 9 = 0$$

Insert 36 at the end of the linked list of location 0.

**Step 6:** Key = 54

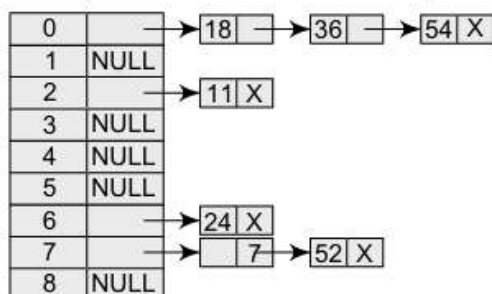
$$h(k) = 54 \bmod 9 = 0$$

Insert 54 at the end of the linked list of location 0.

**Step 7:** Key = 11

$$h(k) = 11 \bmod 9 = 2$$

Create a linked list for location 2 and store the key value 11 in it as its only node.

**Step 8:** Key = 23

$$h(k) = 23 \bmod 9 = 5$$

Create a linked list for location 5 and store the key value 23 in it as its only node.

